1. Introduction to Recommender Systems (RS)

What is a Recommender System?

A **recommender system** is a type of software that predicts the preferences or interests of users and suggests items they are likely to find useful, relevant, or enjoyable.
Examples:

- Netflix suggesting movies or TV shows.
- Amazon recommending products.
- YouTube suggesting videos.

Why are RS important?

- **Information Overload**: Online platforms (e.g., e-commerce, social media, streaming services) contain massive amounts of content. Users often find it difficult to decide what to watch, buy, or read.
- **Personalization**: RS provides tailored recommendations that improve user satisfaction.
- **Business Benefits**: Helps companies increase engagement, sales, and customer loyalty.

---

2. Challenges in Recommender Systems

1. **Robustness**
    - The system should continue to provide accurate results even when data is noisy, incomplete, or unexpected.
2. **Fairness**
    - RS must provide unbiased recommendations and not discriminate against certain groups of users based on demographic characteristics (e.g., gender, race, age).
3. **Data Bias**
    - If the training data itself is biased, the RS may produce unfair or inaccurate results. For example, if most reviews come from one demographic, the RS may favor that group's preferences.

---

3. Types of Recommender Systems

3.1 Collaborative Filtering (CF)

- **Idea**: "People who are similar to you liked this item, so you might too."
- Uses the behavior of a group of users to make recommendations.

Categories of CF:

1. **Memory-Based CF**
    - Uses historical data directly (user ratings, purchase history).
    - Finds similarities between users (user-based) or between items (item-based).
    - Similarity metrics: Pearson Correlation, Cosine Similarity, Jaccard, MSD, PIP, PSS.
    - The algorithm to find similarity is comprised of two key steps: first, the similarity between users or items is calculated using a nearest neighbor-based method; and second recommendations are generated for users based on the sorted similarity values.

    Memory-based CF works in **two phases**:

    1. **Calculate similarity**: Use one of the metrics above to compute similarity between the target user and all other users (or between items).
    2. **Generate recommendations**: Sort users (or items) by similarity, pick the top-N nearest neighbors, and recommend the most popular items among them.

    This is called a **nearest-neighbor-based method** because it relies on "neighbors" (most similar users/items).

    Issues:

    Sparsity of data: In real-world datasets, most users only interact with a tiny fraction of items. The user-item matrix is very sparse, making similarity calculations unreliable.

    High computational cost: To recommend for one user, the system may need to compute similarities with millions of other users/items, which is expensive.

    - Improvements: Use embeddings (e.g., prefs2vec, graph-based embeddings).
2. **Model-Based CF**
    - Uses machine learning models to predict user preferences.
    - Examples: Matrix Factorization, Neural Networks (MLP, CNN, LSTM, RNN, Auto-Encoders).
    - Neural CF models (e.g., Neural Collaborative Filtering, Neural Network Matrix Factorization) learn hidden (latent) relationships between users and items.
3. **Context-Aware CF**
    - Considers extra factors like time, location, mood, or weather.
    - Example: Recommending different music in the morning vs. evening.
    - Challenge: Context data is often complex (hundreds of sensor variables).
    - Solution: Dimensionality reduction techniques (e.g., Autoencoders).

3.2 Content-Based Filtering (CB)

- **Idea**: "If you liked this item, you will like similar items."
- Uses **features of items** and a **user profile** to make recommendations.
- Focuses on user's past interactions and item attributes (e.g., genre, description, keywords).

Techniques:

- **Vector Space Model (VSM)**: Items and user profiles represented as weighted term vectors (TF-IDF weighting).
- **Similarity Measures**: Cosine similarity, Pearson, Kendall-Tau, Spearman.
- **Semantic Analysis**: Uses knowledge bases (WordNet, ontologies) to capture deeper meanings beyond keyword matching.

Examples:

- Systems like Letizia, WebWatcher, Amalthea (for browsing).
- News recommenders like NewsDude, NewT, Daily Learner.

3.3 Knowledge-Based RS

- Does not rely on past user-item interactions.
- Used in domains where purchases are **rare or complex** (cars, medical systems, real estate).
- Relies on **domain knowledge** about items, users, and their interactions.

What is Domain Knowledge?

- **User knowledge**: Needs, constraints, preferences (e.g., "I want a family car with good mileage").
- **Item knowledge**: Characteristics of items (e.g., car type, engine size, price, brand).
- **Interaction knowledge**: Rules that connect users and items (e.g., "If the user has 3 kids, recommend cars with 5 seats or more").

Advanced Models:

- Knowledge Graphs for intent-based recommendations.
- Graph Neural Networks (KCGN, KHGT, KGIN).
- Medical RS (e.g., Safe Medicine Recommendation using EMR + knowledge graphs).

Knowledge-aware Coupled Graph Neural Network (KCGN):

- Uses a **graph structure** (nodes = users/items; edges = relationships).

- Captures **high-order relationships** (e.g., user → item → brand → category).
- Uses **mutual information** to improve awareness of the entire structure, not just direct links.
- Example: If many users interested in "hybrid cars" also searched for "charging stations," the model learns that these concepts are related.

3.4 Hybrid RS

- Combines different techniques (CF, CB, knowledge-based, context-aware).
- Example: Netflix uses both CF (similar users) and CB (movie features).
- Deep learning helps integrate multiple data sources (content, context, user history).

3.5 Group RS

- Provides recommendations for a **group of users** (e.g., friends deciding on a movie).
- Balances different user preferences.
- Useful for collaborative activities (travel planning, group dining).

---

4. Evaluation of Recommender Systems

4.1 Rating-Based Indicators (RBI)

- Evaluate the **accuracy of predicted ratings**.
- RBI evaluates **how well a recommender system predicts rating scores**.
- The quality is measured by comparing the **predicted rating** (r^) against the **true rating** (r).
- Metrics:
  - **RMSE (Root Mean Squared Error)** → penalizes large errors more.
  - **MAE (Mean Absolute Error)** → average absolute difference.
- Lower values = better performance.
- Sensitive to outliers.

4.2 Item-Based Indicators (IBI)

- Evaluate the **quality of recommendation lists**.
- IBI evaluates **how good the system's recommended item list is**.
- **Basic Metrics**: Precision, Recall, F1-score.

- **Ranking Metrics**:
  - **Precision**
    - Out of all items recommended, how many were relevant?
    - Example: If 10 movies were recommended and 6 were actually relevant, precision = 6/10 = 0.6.
  - **Recall**
    - Out of all relevant items in the system, how many did we recommend?
    - Example: If there are 8 relevant movies in total and we recommended 6 of them, recall = 6/8 = 0.75.
  - **F1-Score**
    - Harmonic mean of precision and recall.

THE CONFUSION MATRIX FOR THE RECOMMENDER SYSTEM.

	Recommended	Not Recommended
Used	True Positive (TP)	False Negative (FN)
Not Used	False Positive (FP)	True Negative (TN)

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F\text{-}Measure = \frac{2 * Precision * Recall}{Precision + Recall}$$

  - **Precision@k, Recall@k**
    - Looks only at the **top-k items** in the list (e.g., top 10 recommendations).
  - **MAP@k (Mean Average Precision)**
    - Considers both **relevance** and **ranking order**. Rewards systems that put relevant items higher.
  - **MAR@k (Mean Average Recall)**
    - Measures how many relevant items appear in the top-k list across all users.

    - **NDCG (Normalized Discounted Cumulative Gain)**
        - Handles graded relevance (not just binary). Items at higher ranks are given more importance.
    - **NDPM**
        - Evaluates how well the ranking order of items matches the user's preference order.

Limitations:

- MAP/MAR assume binary relevance.
- NDCG improves by supporting multiple relevance levels.

---

5. Content-Based RS in Detail

Item Representation

- **Structured Data**: Features with well-defined values (e.g., price, category).
- **Unstructured Data**: Text from reviews, product descriptions, articles.
- Problem: Keyword-based matching struggles with semantics and ambiguity.

Improving Content-Based RS

1. **TF-IDF + Vector Space Models**
    - Terms weighted by importance in a document compared to the whole corpus.
    - Similarity measured using cosine similarity.
2. **Semantic Analysis (Ontologies, Lexicons)**
    - Ontologies: Formal definitions of concepts and relationships.
    - WordNet: Groups words into synonym sets and links them by meaning.
    - Helps capture deeper meaning beyond string matching.

Example Systems

- **Web Browsing RS**: Letizia, WebWatcher, Amalthea.
- **News RS**: NewT, NewsDude, YourNews.
- **Ontology-based RS**: SiteIF, ITR, SEWeP.

---

6. Trends in Recommender Systems

- **Neural Approaches**: CNNs, LSTMs, RNNs, Auto-Encoders.
- **Embedding Techniques**: prefs2vec, graph-based embeddings.

- **Generative AI in RS**: LLMs for personalized health recommendations, knowledge graph intent modeling.
- **Context Integration**: Using IoT devices, mobile sensors for more accurate real-time RS.
- **Hybrid & Group RS**: More research in combining methods and handling group decision-making.